

Hybrid Isolation Forest + XGBoost: Training Conclusion

A step-by-step account of the two-stage model. Stage 1 is an Isolation Forest that decides normal vs attack; only flows it calls attack go to stage 2, an XGBoost that names the attack type. This report covers the data, every parameter, how each stage was trained and tested, and what the results mean.

Stage-1 accuracy 83.2% normal vs attack	Stage-1 ROC-AUC 0.865 normal/attack separation	Hybrid accuracy 89.1% 15-class, end to end	Attacks passed on 82.9% reach stage 2	Classes >= 95% 4 / 15 hybrid recall
--	---	---	--	--

Headline. The two-stage model reaches 89.1% overall accuracy. Stage 1 is the gate: its Isolation Forest separates normal from attack to ROC-AUC 0.865 and passed 82.9% of attacks on to stage 2, missing 45,378 (called normal). Section 7 explains the trade-off and how to raise it.

01 — ARCHITECTURE

Two stages, one gate

Each flow passes through at most two models:

- Stage 1 - Isolation Forest (unsupervised).** Scores how unusual the flow is. Below the threshold it is called **normal** and the answer is Benign - the flow stops here. At or above the threshold it is called **attack** and passed on.
- Stage 2 - XGBoost (supervised, 15 classes).** Only for flows stage 1 flagged. It names the attack type, or answers Benign to cancel a stage-1 false alarm.

Consequence of the gate: an attack that stage 1 calls normal never reaches stage 2, so it is lost for good. The hybrid can therefore never catch more attacks than stage 1 lets through.

02 — DATA & SPLIT

Same rows and split for both stages

Source: **cicids2018_full_cleaned.csv** (11,872,582 cleaned flows, 79 columns). Every attack row is used, plus a random 1,000,000 Benign rows (the full 10.5M do not fit in 12 GB of RAM). The CSV is read in 200,000-row chunks and saved as NumPy arrays, so later steps memory-map them and never re-read the 4.8 GB file.

Each class is split **80% train / 20% test** (stratified, seed 42). There is **no validation set**. Both stages use the same split, so the 465,837 test rows are unseen by either model.

Working set: 2,329,184 rows -> train 1,863,347, test 465,837. The Isolation Forest is fitted on the **Benign training rows only**; XGBoost is trained on all 15 classes of the training split.

03 — STAGE 1: ISOLATION FOREST

How the anomaly detector is built

An Isolation Forest builds random trees that repeatedly split the data on a random feature at a random value. Normal points sit in dense regions and take many splits to isolate; unusual points are isolated in a few splits. The average number of splits becomes an **anomaly score** (higher = more unusual). It is trained on Benign traffic only, so it learns what normal looks like and flags what does not fit.

Feature preparation (both fixes were essential - the first version without them flagged 95% of normal traffic):

- **Drop constant columns.** The 8 all-zero columns carry no split information; 70 of 78 features remain.
- **log1p compression.** Rates run 0 to 1.8 billion; without it almost every random split lands in empty space. $\log_{1p}(x + 1)$ is applied ($x \geq -1$ here, so it is always finite).
- **StandardScaler.** Each feature is centred to mean 0 and scaled to variance 1, using statistics from the Benign training rows. This is the single biggest lever - it stops large-range columns dominating the splits.

Parameters:

Parameter	Value	Meaning
n_estimators	300	number of isolation trees; 100-200 is the usual sweet spot
max_samples	4,096	rows drawn to build each tree (larger separates dense floods better)
contamination	0.02	expected share of anomalies; low value, from the IDS literature
max_features	1.0	each split may use any of the 70 features
features used	70	78 minus the 8 all-zero columns
transform	log1p then StandardScaler	applied before scoring
trained on	Benign training rows only	novelty detection
random_state	42	reproducible

Threshold. The score threshold is set to the point of maximum **balanced accuracy** (normal accuracy + attack recall, i.e. Youden's J) on the training split, so stage-1 accuracy is the best this Isolation Forest can give. Chosen value: **0.4644**.

```
score = -score_samples(scaler.transform(log1p(features + 1))) # higher = more unusual
flow is ATTACK if score >= 0.4644 else NORMAL (Benign)
```

04 — STAGE 1 RESULTS

Isolation Forest on the 465,837-row test set

Normal accuracy 83.5%, attack recall 82.9%, balanced accuracy 83.2%. ROC-AUC is **0.865** - the separation quality independent of the threshold (0.5 = useless, 1.0 = perfect). 0.86 means fair but not strong separation.

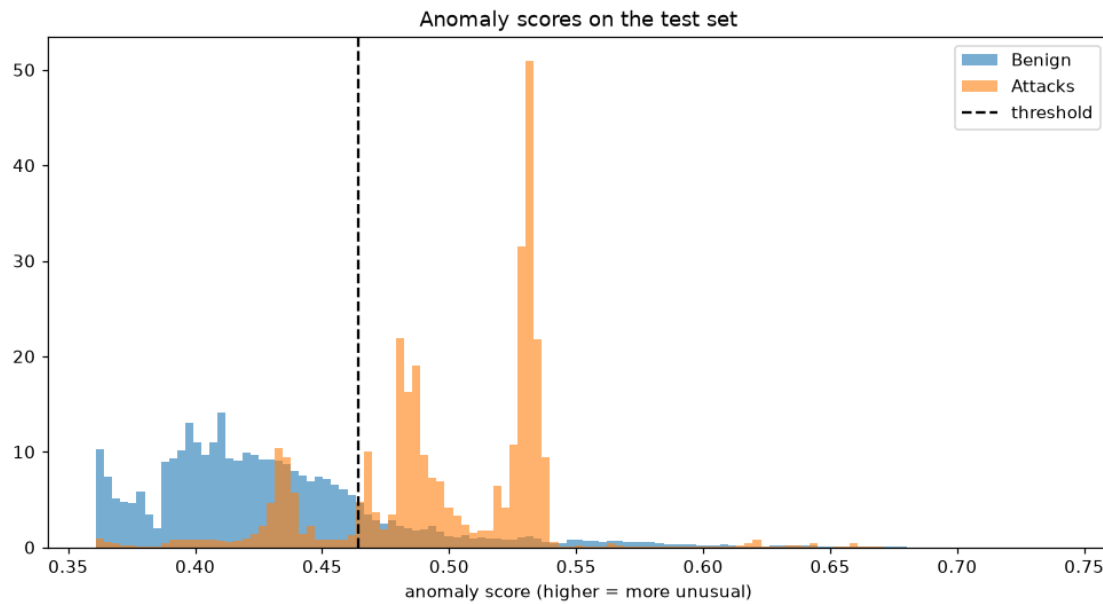
class	test rows	rate	meaning
Benign	199,922	83.5%	normal not flagged
DDoS attacks-LOIC-HTTP	115,073	93.3%	attacks caught
DDoS attack-HOIC	39,772	82.5%	attacks caught
DoS attacks-Hulk	29,040	55.8%	attacks caught
Bot	28,907	99.1%	attacks caught
Infiltration	23,520	37.9%	attacks caught
SSH-Bruteforce	18,808	100.0%	attacks caught
DoS attacks-GoldenEye	8,281	64.2%	attacks caught
DoS attacks-Slowloris	1,982	100.0%	attacks caught
DDoS attack-LOIC-UDP	346	100.0%	attacks caught
Brute Force -Web	109	65.1%	attacks caught
Brute Force -XSS	45	48.9%	attacks caught
SQL Injection	15	53.3%	attacks caught
DoS attacks-SlowHTTPTest	9	100.0%	attacks caught
FTP-BruteForce	8	100.0%	attacks caught

Green = 95%+. Six of 15 reach it. The misses are the dense floods (Hulk 55.8%, GoldenEye 64.2%, HOIC 82.5%) and Infiltration (37.9%): they are large and repetitive, so they look normal to an anomaly detector.

Threshold trade-off — there is no setting that gives 95% on both sides:

false-alarm ceiling	normal accuracy	attack recall
1%	99.0%	1.1%
2%	98.1%	1.3%
5%	95.0%	26.1%
10%	90.0%	57.0%

To keep 95% of normal traffic un-flagged, the Isolation Forest catches only 26% of attacks.



Attack and normal score distributions overlap heavily, which is why no threshold separates them cleanly.

05 — STAGE 2: XGBOOST

The classifier that names the attack

Stage 2 is a gradient-boosted tree classifier (XGBoost), trained on the same 80% split with raw (unscaled) features and all 15 classes. It only ever sees flows stage 1 flagged, and answers with the attack name, or Benign to cancel a stage-1 false alarm.

Parameter	Value	Parameter	Value
<code>objective</code>	<code>multi:softprob</code>	<code>n_estimators</code>	500
<code>learning_rate</code>	0.1	<code>max_depth</code>	10
<code>subsample</code>	0.8	<code>colsample_bytree</code>	0.8
<code>tree_method</code>	<code>hist</code>	<code>sample_weights</code>	<code>sqrt(balanced)</code>
<code>validation / tuning</code>	none (fixed 500 trees)	<code>random_state</code>	42

There is no validation set, so XGBoost uses a fixed 500 trees and no class-weight tuning.

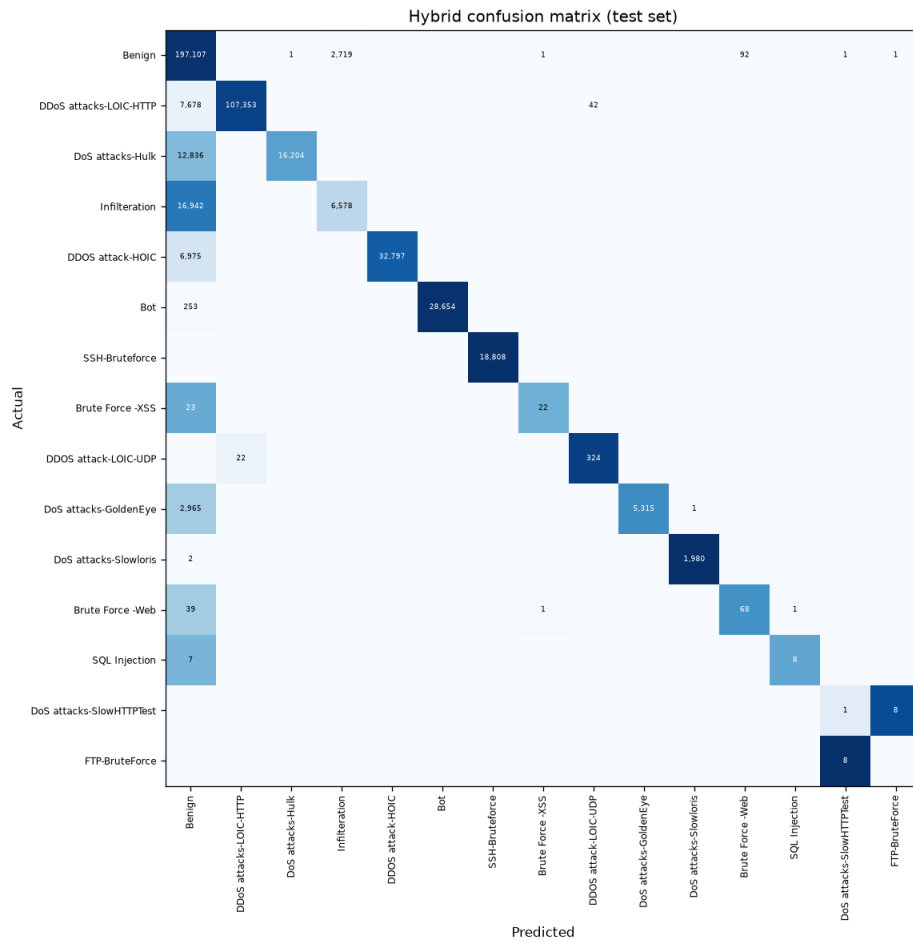
06 — HYBRID RESULTS

End to end on the test set

Stage 1 passed **220,537** attacks to XGBoost and **missed 45,378** (called them normal). It raised 33,006 false alarms, of which XGBoost cancelled 30,191.

class	test rows	precision	recall	F1
Benign	199,922	80.5%	98.6%	0.886
DDoS attacks-LOIC-HTTP	115,073	100.0%	93.3%	0.965
DDoS attack-HOIC	39,772	100.0%	82.5%	0.904
DoS attacks-Hulk	29,040	100.0%	55.8%	0.716
Bot	28,907	100.0%	99.1%	0.996
Infiltration	23,520	70.8%	28.0%	0.401
SSH-Bruteforce	18,808	100.0%	100.0%	1.000
DoS attacks-GoldenEye	8,281	100.0%	64.2%	0.782
DoS attacks-Slowloris	1,982	99.9%	99.9%	0.999
DDoS attack-LOIC-UDP	346	88.5%	93.6%	0.910
Brute Force -Web	109	42.5%	62.4%	0.506
Brute Force -XSS	45	91.7%	48.9%	0.638
SQL Injection	15	88.9%	53.3%	0.667
DoS attacks-SlowHTTPTest	9	10.0%	11.1%	0.105
FTP-BruteForce	8	0.0%	0.0%	0.000
Overall	465,837		89.1%	

Recall = attacks caught by stage 1 AND named correctly by stage 2. Where recall is low (Hulk 55.8%, GoldenEye 64.2%, HOIC 82.5%, Infiltration 28.0%), the loss is at stage 1: those attacks were called normal and never reached stage 2. Where precision is low (Brute Force -Web 42.5%), stage 2 mislabels some flows as that class.



07 — MEASURES USED

What each number means

measure	meaning
Normal accuracy	share of normal flows correctly left un-flagged (1 - false-alarm rate)
Attack recall	share of real attacks flagged / caught (detection rate)
Balanced accuracy	average of normal accuracy and attack recall - the fair stage-1 score
ROC-AUC	how well the score separates normal from attack at ANY threshold (0.5 useless, 1.0 perfect)
Precision	of flows predicted as a class, share that really are it (low = many false alarms)
F1	balance of precision and recall

08 — CONCLUSION

What this shows

- **The two-stage model reaches 89.1% overall accuracy**, with 4 of 15 classes at 95% recall.
- **Stage 1 is honest but limited.** 83.5% normal, 82.9% attack, ROC-AUC 0.865. It cannot reach 95% on both, because the dense DoS/DDoS floods and Infiltration look like normal traffic to an anomaly detector.
- **Stage 1 is the bottleneck.** It missed 45,378 attacks (called them normal), which never reached stage 2, so their recall fell (Hulk, HOIC, GoldenEye, Infiltration). The two-stage result can never exceed what stage 1 passes on.

- **To raise the result, make stage 1 stronger.** A more sensitive threshold passes more attacks but flags more normal traffic; a supervised binary gate (normal vs attack) would pass almost all attacks. The Isolation Forest's own strength is that it can also flag unknown attack types.

Scripts: `xgboost_isolation_forest_training/1_prepare_data.py ... 5_evaluate_hybrid.py`. Every number is from `model_config.json`, `test_metrics.json` and `hybrid_metrics.json`. Compiled 2026-09-29.